

Personal Finance Dashboard

Architecture Documentation (arc42)

Roman Lysser · Module WEBLAB.F2601 · 25. September 2026

1. Introduction and Goals

1.1. Requirements Overview

1.2. Quality Goals

1.3. Stakeholders

2. Architecture Constraints

3. System Scope and Context

3.1. Business Context

3.2. Technical Context

4. Solution Strategy

5. Building Block View

5.1. Level 1: Whitebox Overall System

5.2. Level 2: Backend Architecture

5.3. Level 2: Frontend Architecture

5.4. Conscious Architectural Deviations

6. Runtime View

7. Deployment View

8. Cross-Cutting Concepts

9. Architecture Decisions (ADRs)

9.1. ADR-1: “Uncategorized” is the absence of a category, not a reserved category row

Status: accepted · deviates from the wording of M5 and M6 in the proposal.

9.1.1. Context

M5 lets a transaction be entered without choosing a category, M6 lets a category be deleted without leaving its transactions unassigned. The proposal describes both with a *reserved catch-all category* that such transactions fall into.

A reserved category would be an ordinary row in category with special rules: created with every household, not renamable, not deletable, not selectable as a transfer target of itself, excluded from the suggested onboarding list, and recognisable by a flag or a magic name in every query that treats it differently.

9.1.2. Decision

`transaction.category_id` is nullable. `null` is the catch-all: a transaction without a category belongs to the bucket “Uncategorized”. The foreign key uses `on delete set null`, so deleting a category moves its transactions into that bucket in the same statement, unless the user picks another category to transfer them to.

The bucket is a first-class value everywhere a category appears: the transaction list and the analytics chart label it “Uncategorized”, the history filter offers it as an option (`categoryId=none`), and the category statistics group it as its own slice.

9.1.3. Consequences

- M5 and M6 hold as the user experiences them: entry is never blocked, and deleting a category offers “move to Uncategorized” or “move to another category”. No transaction is ever without a visible assignment.
- No special-case rows: nothing to seed, protect, migrate or filter out of the category management UI. The database enforces the fallback by itself.
- The bucket cannot be renamed; its label comes from the translation files.
- Limits are keyed by (`category_id`, `month`), so uncategorized spending cannot get a limit of its own. It still counts as an expense in the monthly cashflow and therefore lowers Safe-to-Spend. No Should-have story asks for a limit on it.
- Should a limit on uncategorized spending become a requirement, the migration is mechanical: insert one reserved category per household and update the `null` rows to it.

9.2. ADR-2: A new month takes over the nearest earlier limits, automatically only for the current and next month

Status: accepted · deviates from the wording of S4 in the proposal.

9.2.1. Context

S4 asks that a new month takes over the limits of the *previous month*, can be changed there, and leaves the previous month unchanged, so that limits need not be entered again every month.

Three questions are left open by that sentence:

- What counts as the previous month when that month has no limits, for example because the app was not used in August?
- When is a month “new”? The system has no scheduler, and a month nobody opens needs no limits.
- What happens to a month the user emptied on purpose? S1 lets every limit be removed, S3 lets a month stay deliberately unbudgeted. A rule that refills every empty month undoes both.

9.2.2. Decision

- **Copies, not a fallback.** Taking over inserts independent budget rows for the target month. Reading a month never looks at other months, so editing a copy cannot change its source, and “no row” keeps meaning “no limit” (S3).
- **Source is the nearest earlier month that has limits**, not strictly the calendar month before. One unused month does not lose the plan.
- **Automatic only for the current and the next calendar month**, on first view (POST `.../:month/copy-previous?auto=true`). Every other empty month offers an explicit “Take over previous limits” action. The target month must be empty (409 otherwise).

- **Touched months never fill themselves again.** Table `budget_month(household_id, month)` records every month whose limits were set, removed or taken over. The automatic take-over skips such a month, so removing the last limit leaves it empty. The explicit action still fills it.

9.2.3. Consequences

- S4 holds for the common case: opening a new month shows last month's limits, and changing them leaves that month as it was.
- S1 and S3 hold together with S4: a month emptied on purpose stays empty until the user asks otherwise.
- A month nobody opened is not filled and costs nothing; past months are filled only on request.
- Skipping is decided per month only. If August was emptied on purpose, September still takes over the limits of July, the nearest month that has any. Treating an emptied month as "inherit nothing" would help this edge case and lose the plan after every unused month, so it was rejected.
- The trigger is the client clock (current/next month), since the server knows no household timezone. A client with a wrong clock can at most fill a month early; the user can remove the copies.
- `budget_month` rows are never deleted on their own; they go with the household.

10. Quality Requirements

10.1. Quality Tree

10.2. Quality Scenarios

11. Risks and Technical Debt

12. Glossary